

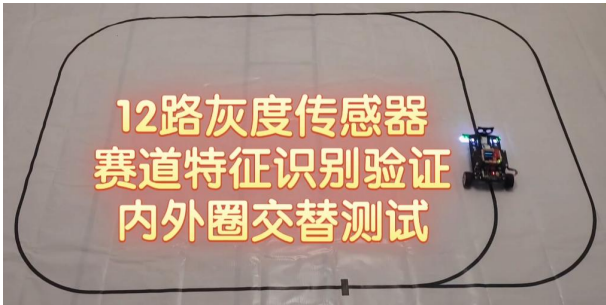
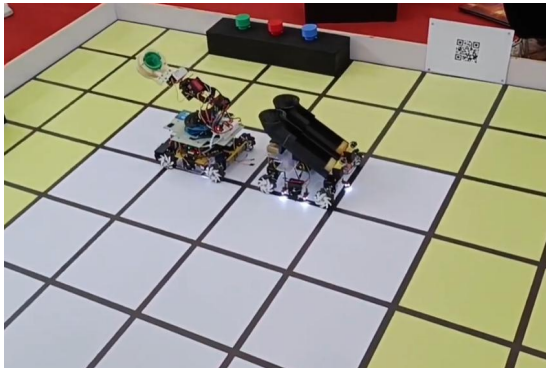
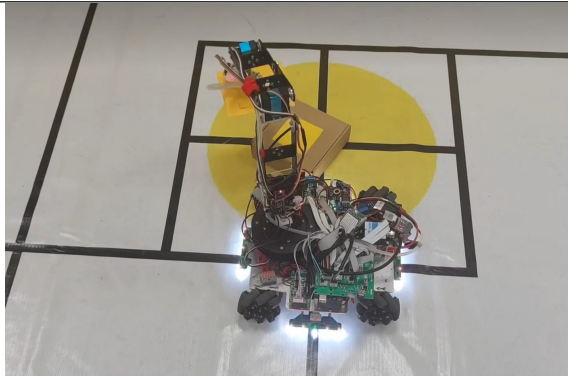
灰度传感器用户使用手册

基于 NCHD1（迹）

- 1、单路可级联数字/模拟双输出灰度传感器讲解
- 2、电源级联 NCHD1 实现多路线性分布的灰度传感器阵列方法
- 3、利用 NCHD1 配合扩展板实现多路灰度传感器阵列讲解

产品用途:

- 灰度传感器主要是在智能车自主循迹、赛道特征识别、距离接近开关等情况中使用，该产品具有灵敏度强，抗干扰能力强，抗黑暗/普通灯照射环境等特点。本产品使用一对抗干扰较强的光电传感器，其中发射管的光源采用高亮白色聚光 LED，发射管端发出的光线通过不同环境背景的反射之后，最终由光敏接收管来接收，光敏接收管的阻抗随反射光线的强弱变化而变化（反射光线越强，阻值越小），最后通过分压和运放比较电路实现数字/模拟信号的双输出。灰度传感器模块对白光反射强弱不同背景环境有非常好的识别效果，背景差异越大，分辨效果越好。灰度传感器相比普通红外传感器有着更高的抗干扰能力。

灰度传感器在相关竞赛中使用的场景	大学生电子设计竞赛	
	工程实践与创新能力大赛练大赛搬运机器人	
	中国工程机器人大赛搬快递配送、搬运机器人等	

- 通过多个灰度传感器能组成灰度传感器阵列，能高效、快速、精准的识别出赛道中线的位置，获取智能车车体与赛道中线偏移程度，小车控制器就可以通过设计一组转向 PID 控制器对智能车运动方向进行实时的调整，从而实现自主循迹，将底部引导线死死的“咬住”。
- 通过灰度传感器阵列分布规律，并结合前后检测到的灰度管输出状态、持续时间等信息，可以用于识别起始线、分叉路、十字路口、直角路口等赛道特征信息，用于辅助决策智能车的运动控制。
- 由于不同反射距离，光敏管收到的光线强度不一样，有效反射距离内，距离越大接收到的光线越小，可以通过这一原理，把灰度传感器模块作为距离接近开关使用，例如智能车竞赛中坡道、障碍的检测。

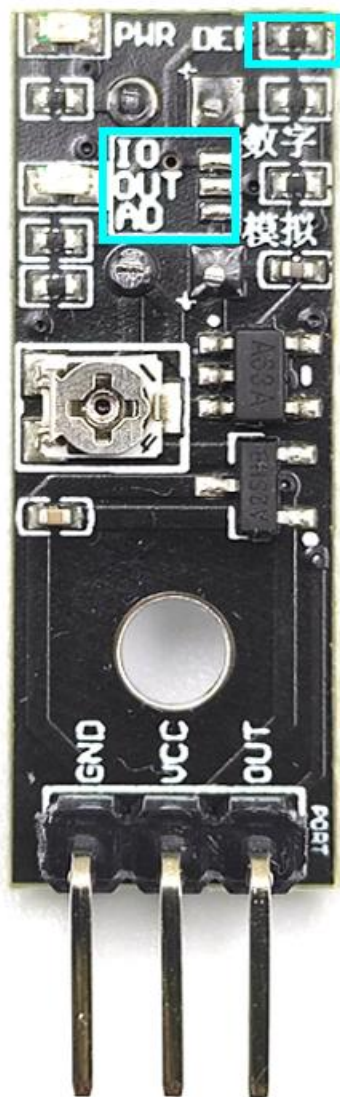
产品参数与主要特点:

- 多种输出接口：2.54mm 3P 排针/XH2.54 3P 母座



- **支持两种信号输出形式**：数字(默认)、模拟，输出方式通过模块上的“输出模式选择区”的焊盘短接来选择；

数字信号和模拟信号输出配置介绍



零欧电阻

输出模式选择区

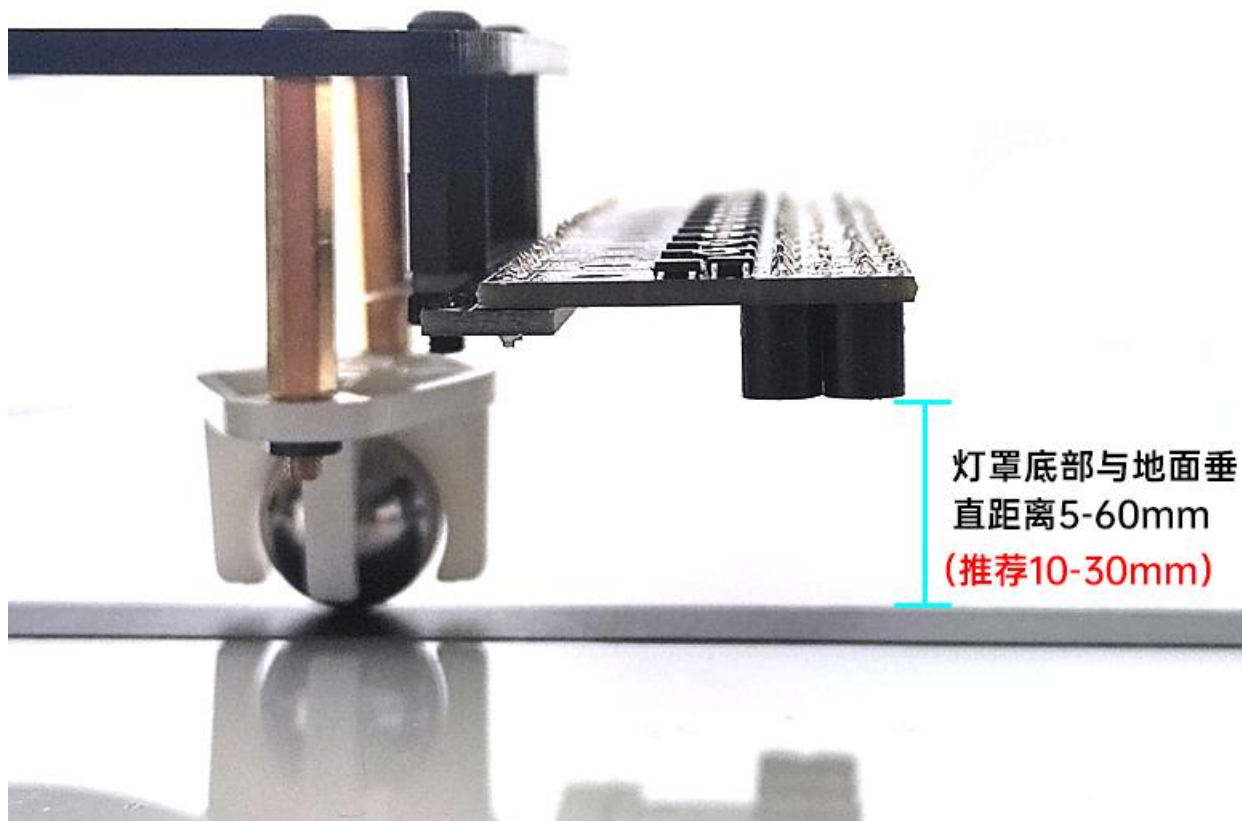
1. 出厂板上焊有零欧电阻，默认输出数字信号(高低电平)

2. 需要配置为模拟输出时，首先取下DEF处的零欧电阻，然后将输出模式选择区中的OUT和AD焊盘短接

3. 当已取下DEF处的零欧电阻后，仍然需要配置为数字输出时，只需将输出模式选择区中的OUT和IO焊盘短接

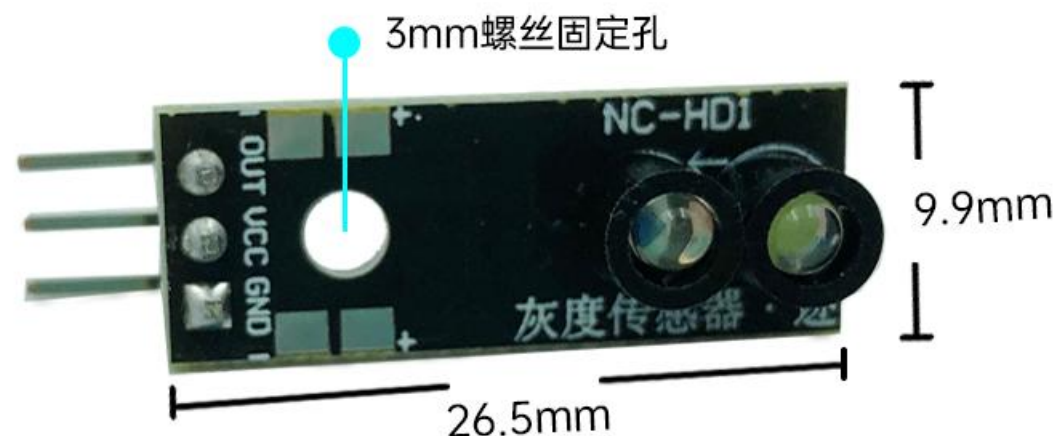
注意：不要将输出模式选择区中的三个焊盘全部短接！！

- 探测距离(灯罩底部的离地距离):5-60mm，推荐 10-30mm;



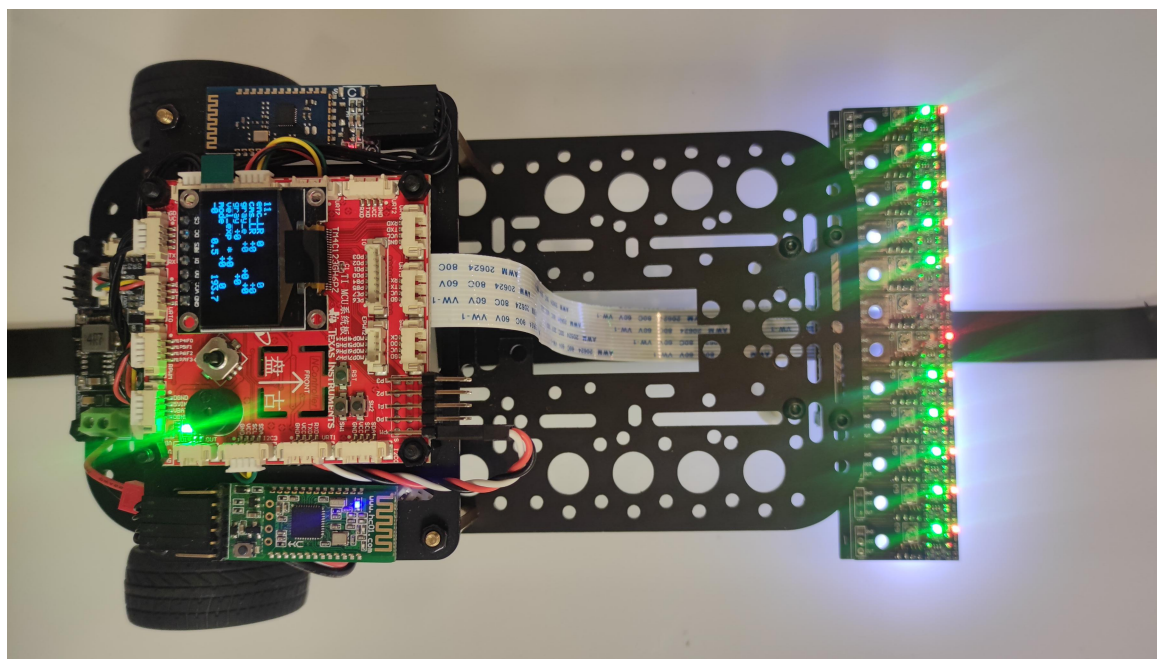
- **带 LED 电源和检测状态指示灯**：红色 PWR 为电源指示灯，绿色 OUT 为状态指示灯，当管子照到灰度值较低的背景，输出电平由低变高，对应 OUT 状态指示灯 LED 灭。(例如在黑色引导线+白色背景使用场景时，照射到黑线时输出高电平 1，指示灯灭；反之照射到白色背景时输出低电平 0，指示灯亮)；
- 模拟电压输出时：
 - 当管子从灰度值较低的背景切换到灰度值较高背景时，模拟电压值增加；
 - 在有效反射高度范围内，当管子从离地高度值由低到高变化时，模拟电压值增加；

- 模块小巧的外形尺寸：宽度 9.9mm、长度 26.5mm，带 3mm 固定孔；



- 带级联电源口，当多个模块电源并联时可以简化整体连线；

- 模块精致小巧方便客户根据不同使用场景自定义扩展线性灰度传感器阵列（如样车中 12 路灰度传感器循迹模块）；



- 工作电压范围：3.3V 到 5V(推荐使用 5V)
- 工作电流：9 到 12mA；

- 带电压反接保护；
- 带定制遮光灯罩，屏蔽环境光线干扰；
- 进口原装运算放大器、微调电位器用料十足，保证出厂品质；
- 灰度传感器仅依靠单片机 IO 口输入检测，基本上不需要占用处理器额外的资源，能对赛道情况实现超高频率的检测

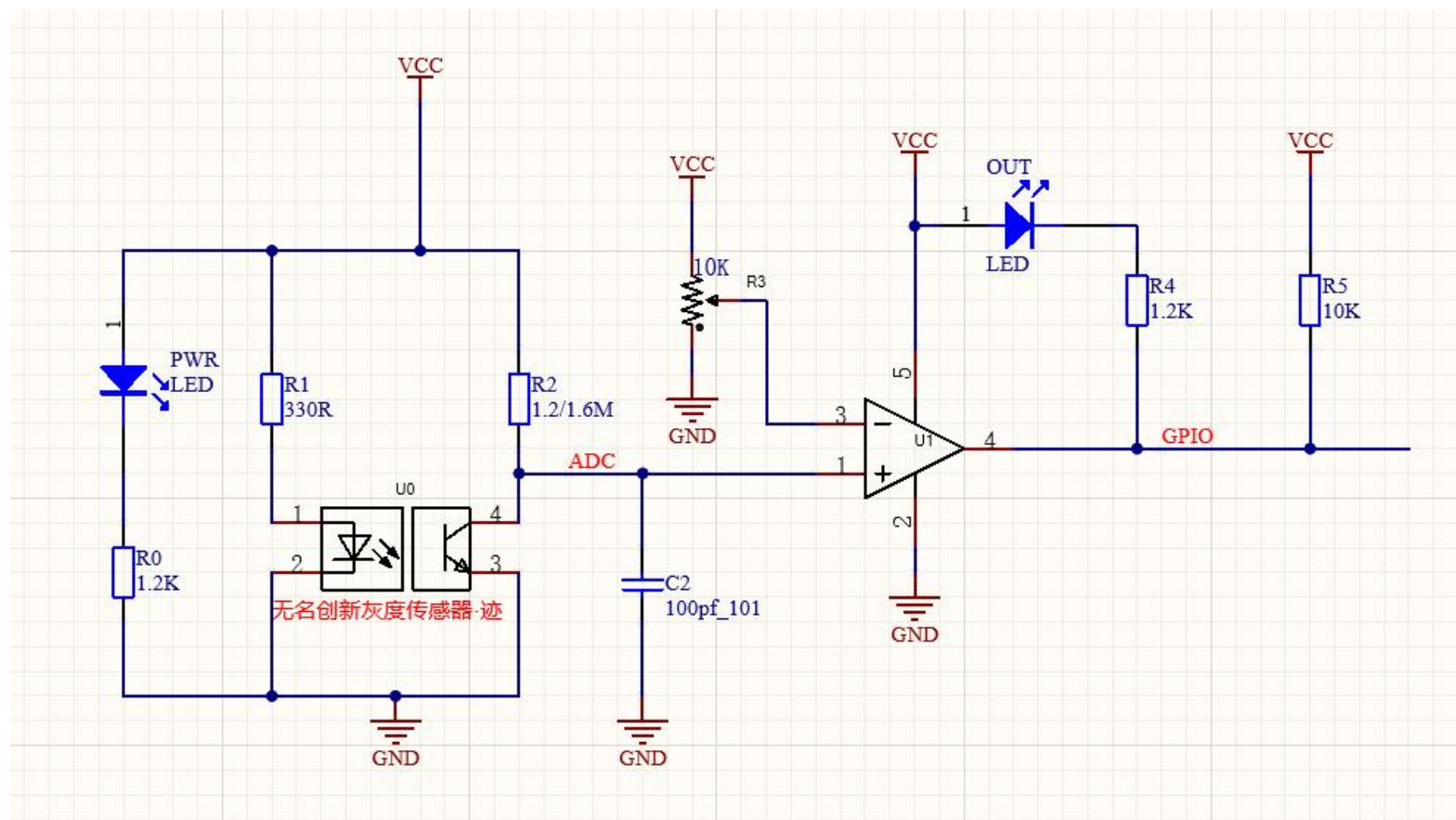
支持多路灰度传感器级联的设计初衷及传统方案中的痛点：

- 在智能小车自主寻迹或者特征检测项目设计中，一般至少需要两路灰度传感器并列排布实现赛道偏移或者特征的检测，市面上已有的 2 路或者多路灰度传感器模块方案**主要存在以下痛点**：
 - 传感器**间距或者分布固定**，不能满足多项比赛场景中的不固定宽度的引导线寻迹、特征检测情况。
 - 用户根据比赛内容**采用友商的单个灰度传感器模组去自定义间距**时，由于灰度传感器**模块偏大**，不能满足引导线小于等于 10mm 的情况，同时由于**不同模块需要重复接电源线**，使得电路连线相对冗杂，降低了整体可靠性。
 - 自己设计灰度传感器线性阵列**不能保证全程对轨迹的检测**，存在中间**检测不到的虚位点**，使得控制器设计时很被动，无法做到流畅的自主寻迹。

- 针对不同比赛要求，以往学生需要重复购买不同厂家的产品，不能重复利用加大了开销。
- 为了方便用户针对不同比赛要求，灵活地对灰度传感器间距、分布进行自主设计，NCHD1 模块宽度为 9.9mm 满足当前几乎所有比赛的引导线要求，用户可以通过背面的电源级联口，实现多路传感器的电源的并联，用户在对多路灰度传感器连接时，只需要对其中一组模块供电就可以实现所有模组的供电，对单个模组只需要接一根信号线，这样的设计大大的简化了连线。
- 用户可以通过 NCHD1 模块，自主设计传感器扩展板方便实现多路传感器线性阵列的设计，精致小巧的设计可以方便客户便捷的取下之前的灰度传感器模组，实现传感器的重复利用。

灰度传感器检测原理:

- 灰度传感器是一组模拟传感器，由一只发光二极管和一只光敏接收管组成，二者安装在同一面上。灰度传感器利用不同颜色的检测背景对光的反射程度不同，同时光敏接收管接收到不同检测面返回的光线，其阻值也不同的原理进行颜色深浅检测。
- 当光敏接收管接收到的光线越强，它的阻值就越小，即地面背景灰度越深或者离地距离越远，光敏二极管的阻值越大。
- 在有效的检测距离内，发光二极管发出白光，照射在检测面上，检测面反射部分光线，光敏接收管检测此光线的强度并通过分压电路和运放比较电路将其转换模拟/数字信号，最后通过机器人主板上的模拟/数字口输入到微控制器进行处理。



- 在上图所示灰度传感器检测电路中，U0 表示灰度传感器模组，左边 12 端的为发射管，右边 34 端的为光敏接收管。

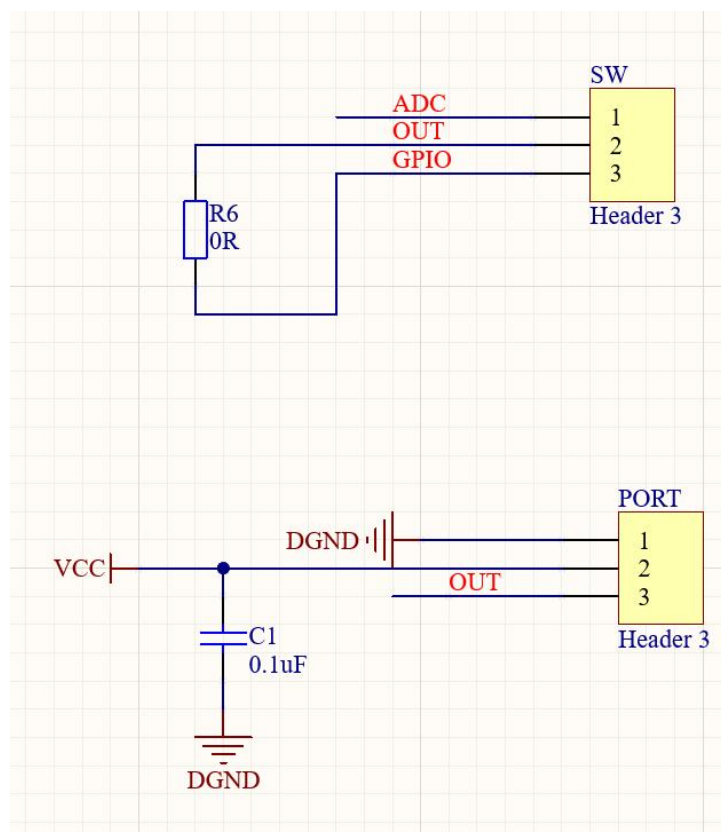
- 当地面背景为深色或者高灰度值时，经地面反射后进入光敏接收管端的光线较少，34 端输出阻抗 RO 很大计为 RO_max ， RO_max 和 $R2$ 组成的分压电路会在 ADC 端输出较大的模拟电压值 $VCC*RO_max/(RO_max+R2)$ 计为 $V+_max$;
- 当地面背景为浅色或者低灰度值时，经地面反射后进入光敏接收管端的光线多，34 端输出阻抗 RO 很小计为 RO_min ， RO_min 和 $R2$ 组成的分压电路会在 ADC 端输出较小的模拟电压值 $VCC*RO_min/(RO_min+R2)$ 计为 $V+_min$ ，由数值比较关系显然有 $V+_max > V+_min$;
- ADC 端的电压经过 $C2$ 电容滤波后送入运算放大器 $U1$ 的+端，运算放大器的-端接入的是有 $R3$ 电位器分压而来的比较电压 $V-com$ ，由电压比

较器原理可知，当比较电压 V_{-com} 处于 V_{+_max} 和 V_{+_min} 之间时（一般取二者中值）即 $V_{+_max} > V_{-com} > V_{+_min}$ ，就可以实现对两种反射背景实现区分；

- 由电压比较电路工作原理可知，灰度传感器照射到深色背景时 $V_{+_max} > V_{-com}$ ，电压比较器+端输入大于-端输入，电压比较器输出高电平，即深色背景时 GPIO 端输出高电平 1，OUT 状态指示灯灭；反之灰度传感器照射到浅色背景时 $V_{+_min} < V_{-com}$ ，电压比较器+端输入小于-端输入，电压比较器输出低电平，即浅色背景时 GPIO 端输出低电平 0，OUT 状态指示灯亮。

数字信号和模拟信号输出选择:

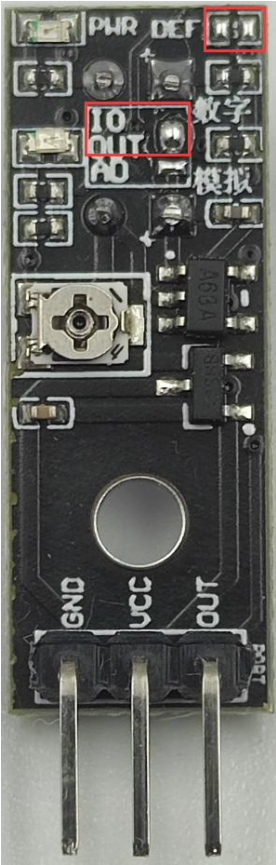
- NCHD1 灰度传感器能输出数字量和模拟量两种信号;



- 灰度传感器模块输出端口 OUT 可以通过 SW 端口实现对接入数字信号

GPIO 与模拟信号 ADC 的选择,同时考虑到大多数用户使用场景为数字信号,故本模块在出厂设置时采用零欧电阻 R6 实现了默认输出数字信号的设置;

- 当用户需要输出模拟 ADC 信号时,需要使用电烙铁将电阻 R6 取下,并用焊锡短接 ADC 和 OUT 处的焊盘。
- 当用户取下了零欧电阻 R6 后,仍然需要输出数字信号时,用焊锡短接 IO 和 OUT 处的焊盘即可。

出厂默认数字输出	模拟输出设置	数字输出设置
		
默认	取下 DEF 处零欧电阻，并短接 OUT 和 AD 焊盘	取下 DEF 处零欧电阻，并短接 OUT 和 IO 焊盘
NCHD1 模块出厂 DEF 处焊接有零欧电阻 R6，相当于短接了 OUT 和 IO，故模块出厂默认输出数字量		

单路可级联数字/模拟双输出灰度传感器接线方法：

● 数字信号输出时：

NCHD1 传感器	GND	VCC	OUT
控制主板	接主板 GND 负极	接 5V 或者 3.3V 正极	接单片机普通 IO 口上
注意事项		为了增加检测有效距离，充分发挥模块性能， 当单片机 IO 口最大输入耐压值支持 5V 时，推荐 优先使用 5V 供电	需要将该 IO 口初始化为 输入模式，否则单片机 IO 电平状态可能会影响 模块输出

● 模拟信号输出时:

NCHD1 传感器	GND	VCC	OUT
控制主板	接主板 GND 负极	接 5V 或者 3.3V 正极	接单片机带 ADC 转换的 IO 口上
注意事项		当使用的单片机 ADC 采集参考基准电压为 3.3V 时, 请务必使用 3.3V 供电, 若使用 5V 供电, 灰度传感器输出模拟电压值有可能会超过基准电压 3.3V, 从而导致 ADC 数据采集异常	需要将该 IO 口初始化为 ADC 采集模式, 否则单片机 IO 电平状态可能会影响模块输出

灰度传感器 NCHD1 电位器的调节：

- 灰度传感器上的电位器逆时针旋转时，比较电压 V-com 增加，电压变化范围从 0 至 VCC;
- 快速调节电位器方法分三步
 - 在有效高度范围内（5-60mm，推荐 10-30mm），固定好灰度传感器，并将 NCHD1 上电位器顺时针拧到底；
 - 将灰度传感器对着浅色背景，逆时针缓慢拧动电位器，直至绿色输出状态指示灯 OUT 亮起后，并继续逆时针略微拧动一小段（目的避免传感器上下振动、地面不平整造成误判，特意留出来的一定电压裕度，增强检测的鲁棒性）。
 - 将灰度传感器从浅色背景上移至深色背景，绿色输出状态指示灯 OUT 会由亮变灭。在深色背景上抬起或者接近地面，略微调整 NCHD1 和地面的高度，指示灯状态不会发生跳变就表示电位器设置好了。

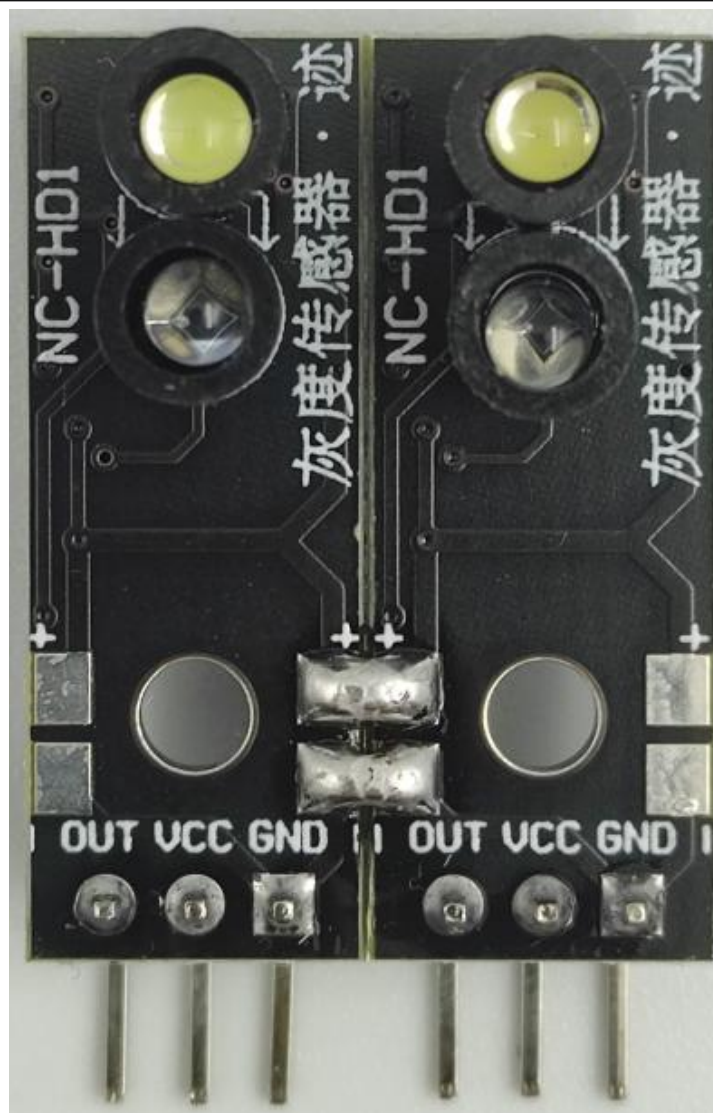
- 为了方便大家记忆，将上述快速调节电位器方法整理成了如下口诀：

高度固定顺拧到底
浅色逆拧待灯亮起
求稳继续微拧少许
移至深色绿灯即灭
上下微抖灯仍不变
反复测试无惧颠簸

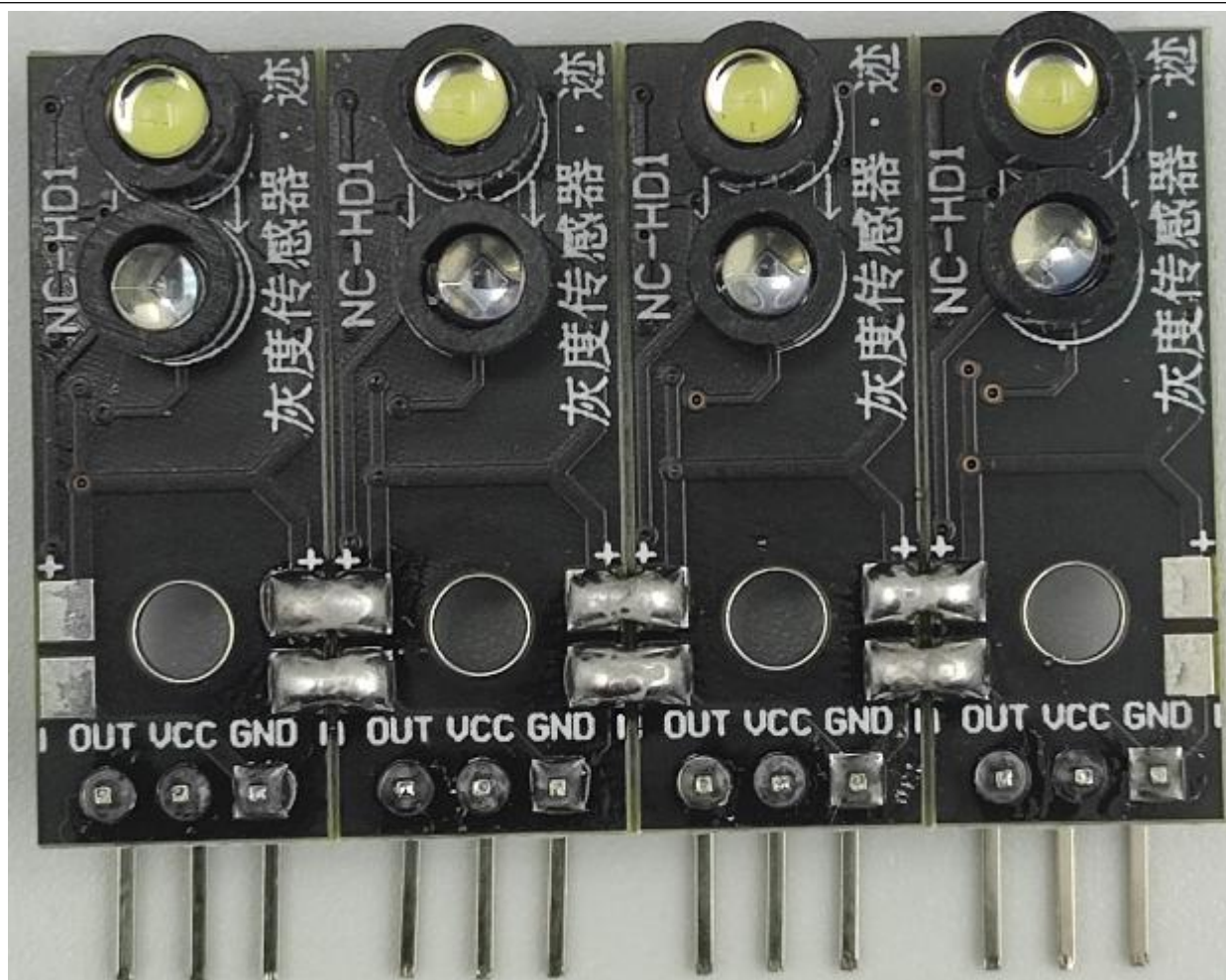
电源级联 NCHD1 实现多路线性分布的灰度传感器阵列方法：

- NCHD1 灰度传感器上背面两侧有两组电源接口，用户可以通过此接口从相邻分布的其它 NCHD1 上获取电源，效果等同于直接从排针处供电。下面以 2 路、4 路、6 路为例子给出焊接实物图。
- 通过焊锡连接背面相邻的电源焊盘，可以实现简化硬件连线，多路灰度传感器线性分布时，只需要给其中一组提供电源就可以，实现对整体模组的供电。

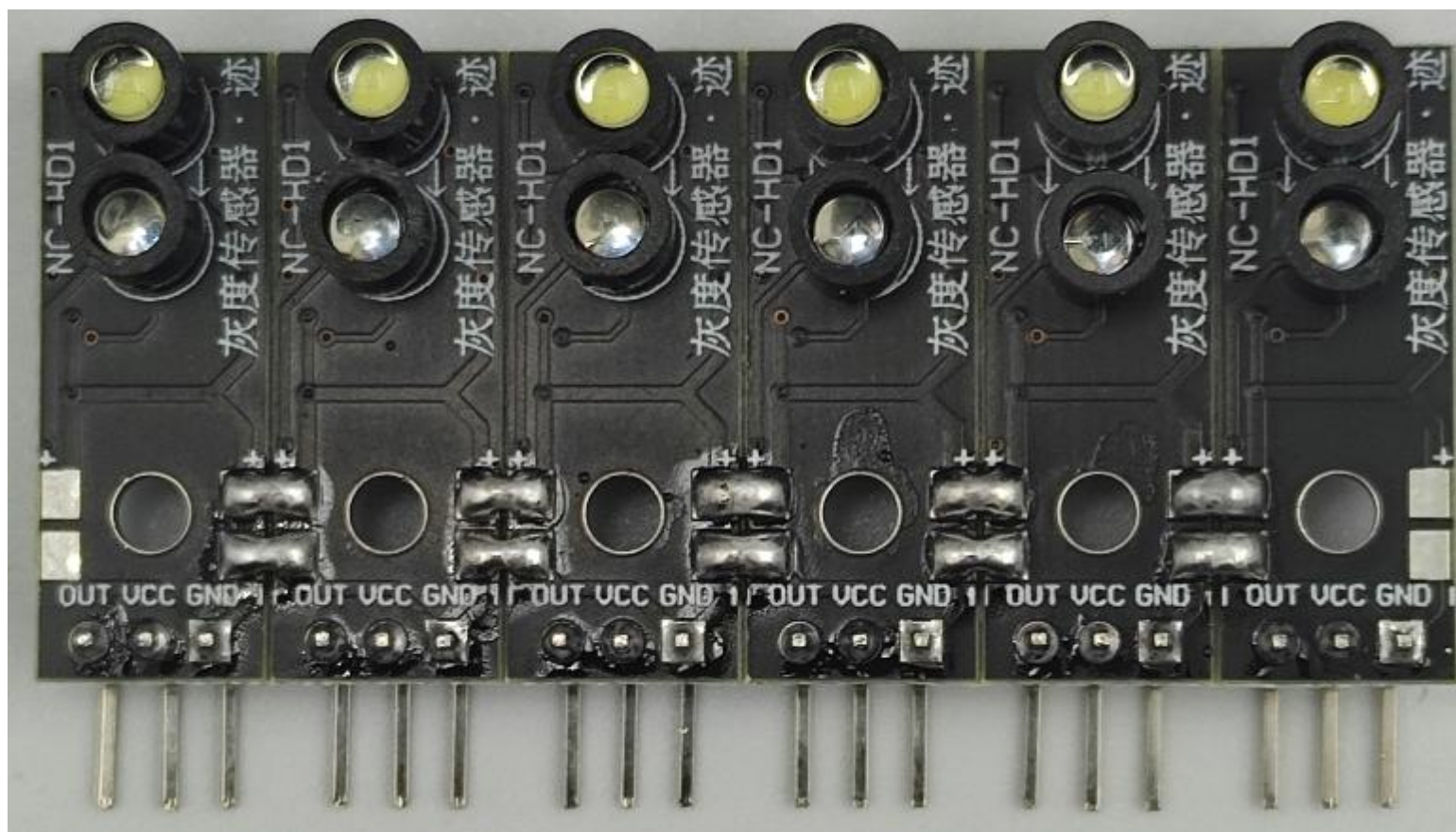
2 路灰度灰
度传感器
电源并联



4 路灰度灰
度传感器
电源并联

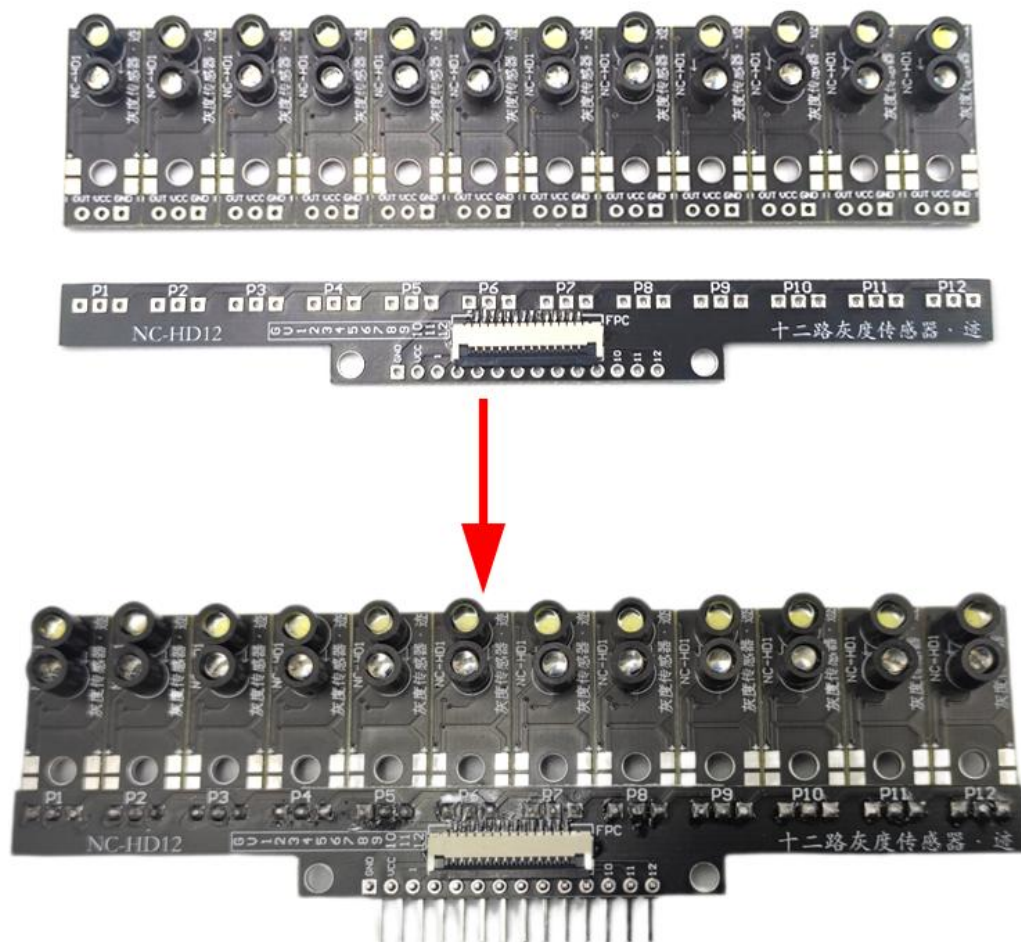


6 路灰度灰
度传感器
电源并联



利用 NCHD1 配合扩展板实现多路灰度传感器阵列讲解：

- NCHD1 灰度传感器尺寸为 9.9mm*26.5mm，精致小巧的结构设计可以使用户很方便地将 NCHD1 当做独立的模组焊接在自己设计扩展板 PCB 上，进而实现任意多路的灰度传感器线性阵列，用户可以根据实际项目中的引导线宽度要求，去合理规划传感器间距、传感器数量、传感器位置分布等细节。高校针对参加电子设计竞赛的实验室基本都配有喷墨打印机、热转印、腐蚀池等制作电路板的设备，学生现场可以高效率的设计出满足比赛要求的多路灰度传感器线性阵列扩展板。



NC-HD1扩展成12路灰度传感器线性阵列示例

常见问题整理合集：

➤ 我用的是 STM32 单片机，单片机工作电压为 3.3V，给传感器模块供电使用 5V 时，模块输出高电平能直接接到单片机 IO 口上吗？

✓ 答：当灰度传感器模块采用 5V 供电时，模块输出高电平时的电压约为 4.0V。对于工作电压为 3.3V 的单片机，一般 IO 口能承受的最大高电平电压为 5V，具体需要客户自己查阅单片机手册，对于 STM32 单片机，手册中功能介绍中标有 FT 的 IO 口都支持 5V 电平接入。对于部分不支持 5V 电平输入的单片机，保险起见可以将模块输出信号串联一个 300-1000 欧姆的电阻后再接入单片 IO 口，或者直接使用 3.3V 给灰度传感器模块供电更为稳妥。

➤ 传感器安装高度距离地面多少合适？

✓ 答：按照不同比赛中客户实际调试经验，对于自主循迹、赛道特征检测应用场景，我们推荐的传感器安装高度为 10~30mm，安装高度是以灰度传感器模块底部的遮光罩为基准的对地高度。客户可以根据实际比赛中引导线宽度、背景颜色、环境光线影响等因素灵活选择传感器的安装高度，核心要点是能稳定检测不误判。

- 传感器模块支持数字信号和模拟信号双输出，我该怎么选择用哪一种？
- ✓ 答：灰度传感器出厂默认输出方式为数字信号，输出信号为高低电平，接入控制器的方法为单片机普通 IO 口输入检测，数字信号输入能满足绝大多数应用场景，且占用单片机资源相对较少，在识别赛道和背景颜色只有两种情况下，推荐客户优先选择数字信号输出的方式。针对实际项目中需要识别的背景颜色有多种的情形，由于模拟信号输出的是连续变化的电压信号，灰度传感器模块照射到不同颜色的背景时，在 ADC 端输出的电压值不一样，客户可以通过单片机采集到的不同的 ADC 电压区间值来判断底部背景颜色，这种情况下就必须采用模拟信号输出。
- 传感器模块不接单片机时工作正常，接入单片机后不管是深色还是浅色背景，绿色输出指示灯都是常亮/灭，这是为什么？
- ✓ 答：需要把单片机对应 IO 口工作方式设置为输入后再接入灰度传感器，否则单片机 IO 口的随机或者输出电平将影响模块正常输出。

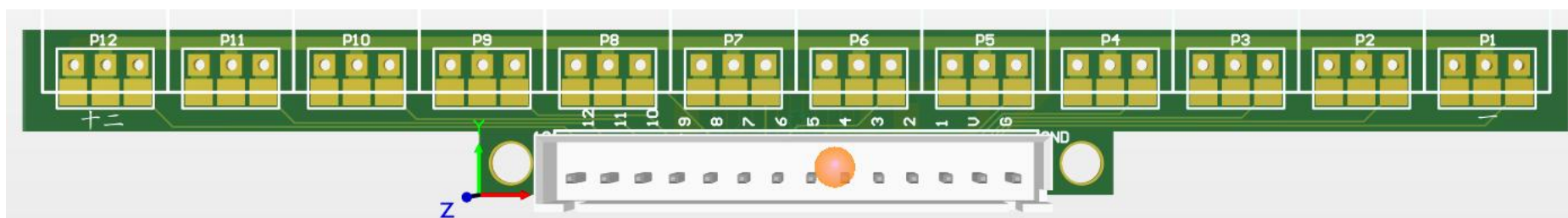
➤ 购买此灰度传感器模块提供哪些相关技术支持和资料？

✓ 答：我们为灰度传感器 **NCHD1** 模块针对初学者用户编写和录制了详细的检测原理、使用方法、注意事项等配套文档和视频教程，能确保新手用户在模块到手后，能快速对模块功能进行测试，并且附送了 **51** 单片机、**STM32** 单片机、**TIVA** 单片机、**MSP432** 单片机、**Arduino** 等处理器平台的单/多路识别检测例程，辅助新手用户快速嵌入到自己的项目中。配套资料通过网盘方式下载，客户直接联系淘宝客服获取。

➤ 在多路灰度传感器组成线性阵列使用场景中，如何将灰度传感器输入状态量化成位置偏差？

✓ 答：下面以 **TIVA** 单片机为处理器，分别以 **7** 路和 **12** 路为例来讲解传感器输入状态到最终偏差的处理过程
首先定义 **16** 位数据来存储 **16** 个灰度传感器每一位状态的结构体变量 **gray_flags**。

```
53  
54 typedef struct  
55 {  
56     uint8_t bit1   :1;  
57     uint8_t bit2   :1;  
58     uint8_t bit3   :1;  
59     uint8_t bit4   :1;  
60     uint8_t bit5   :1;  
61     uint8_t bit6   :1;  
62     uint8_t bit7   :1;  
63     uint8_t bit8   :1;  
64     uint8_t bit9   :1;  
65     uint8_t bit10  :1;  
66     uint8_t bit11  :1;  
67     uint8_t bit12  :1;  
68     uint8_t bit13  :1;  
69     uint8_t bit14  :1;  
70     uint8_t bit15  :1;  
71     uint8_t bit16  :1;  
72 }gray_flags;
```



`gray_flags` 结构体中的为 16 位数据，每一位可以表示一路灰度传感器的状态，为了方便后续处理，我们约定 `bit1~bit16` 分别表示的是传感器阵列的最右端到最左端的灰度传感器高低电平状态。比如以上图中 12 路灰度传感器为例：`bit1` 用于存储 `P1` 的高低电平状态、`bit2` 用于存储 `P2` 的高低电平状态..依次类推。

利用 `gray_flags` 结构体我们定义一个联合体变量 `gray_state`, `gray_state` 变量中 `gray` 结构体变量和 `state` 变量共同占用 2 个字节数据空间, 在实际使用时我们可以通过单片机的 `gpio` 读取函数对 `gray` 变量的每一位赋值, 赋值之后可以通过判断 `state` 数据的值来对偏差值进行量化。

```
8 union
9 {
10     gray_flags gray;
11     uint16_t state;
12 } gray_state;
13
```

变量定义完毕后, 在读取单片机 IO 口高低电平之前需要对 `gpio` 相关资源进行初始化, 相关函数如下:

```
22
23  /*****
24  函数名: void gpio_input_init(void)
25  说明: 12路灰度管gpio检测初始化
26  入口: 无
27  出口: 无
28  备注: 无
29  作者: 无名创新
30  *****/
31  void gpio_input_init(void)
32  {
33      SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOD);
34      SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOE);
35      SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOB);
36      SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOH);
37
38      GPIODirModeSet(GPIO_PORTD_BASE, GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3,GPIO_DIR_MODE_IN);
39      GPIOPadConfigSet(GPIO_PORTD_BASE,GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3,GPIO_STRENGTH_4MA,GPIO_PIN_TYPE_STD_WPD);
40
41      GPIODirModeSet(GPIO_PORTE_BASE, GPIO_PIN_6|GPIO_PIN_7,GPIO_DIR_MODE_IN);
42      GPIOPadConfigSet(GPIO_PORTE_BASE,GPIO_PIN_6|GPIO_PIN_7,GPIO_STRENGTH_4MA,GPIO_PIN_TYPE_STD_WPD);
43
44      GPIODirModeSet(GPIO_PORTB_BASE, GPIO_PIN_5|GPIO_PIN_4,GPIO_DIR_MODE_IN);
45      GPIOPadConfigSet(GPIO_PORTB_BASE,GPIO_PIN_5|GPIO_PIN_4,GPIO_STRENGTH_4MA,GPIO_PIN_TYPE_STD_WPD);
46
47      //PH4 PH5 PH6 PH7
48      GPIODirModeSet(GPIO_PORTH_BASE, GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7,GPIO_DIR_MODE_IN);
49      GPIOPadConfigSet(GPIO_PORTH_BASE,GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7,GPIO_STRENGTH_4MA,GPIO_PIN_TYPE_STD_WPD);
50  }
```

根据前文所述，首先需要读取单片机对应 IO 的高低电平状态，并将电平状态赋值给 gray 变量的对应位，高电平赋值 1，低电平赋值 0。


```
void gpio_input_check_channel_12(void)
{
    gray_state.gray.bit1=GPIOPinRead(GPIO_PORTH_BASE,GPIO_PIN_7)!=0?0x01:0x00;
    gray_state.gray.bit2=GPIOPinRead(GPIO_PORTH_BASE,GPIO_PIN_6)!=0?0x01:0x00;
    gray_state.gray.bit3=GPIOPinRead(GPIO_PORTH_BASE,GPIO_PIN_5)!=0?0x01:0x00;
    gray_state.gray.bit4=GPIOPinRead(GPIO_PORTH_BASE,GPIO_PIN_4)!=0?0x01:0x00;

    gray_state.gray.bit5=GPIOPinRead(GPIO_PORTE_BASE,GPIO_PIN_6)!=0?0x01:0x00;
    gray_state.gray.bit6=GPIOPinRead(GPIO_PORTE_BASE,GPIO_PIN_7)!=0?0x01:0x00;
    gray_state.gray.bit7=GPIOPinRead(GPIO_PORTB_BASE,GPIO_PIN_5)!=0?0x01:0x00;
    gray_state.gray.bit8=GPIOPinRead(GPIO_PORTB_BASE,GPIO_PIN_4)!=0?0x01:0x00;

    gray_state.gray.bit9 =GPIOPinRead(GPIO_PORTD_BASE,GPIO_PIN_0)!=0?0x01:0x00;
    gray_state.gray.bit10=GPIOPinRead(GPIO_PORTD_BASE,GPIO_PIN_1)!=0?0x01:0x00;
    gray_state.gray.bit11=GPIOPinRead(GPIO_PORTD_BASE,GPIO_PIN_2)!=0?0x01:0x00;
    gray_state.gray.bit12=GPIOPinRead(GPIO_PORTD_BASE,GPIO_PIN_3)!=0?0x01:0x00;
```

由于 gray 变量和 state 变量为联合体，我们可以通过判断 state 变量的数值来对位置偏差进行量化。


```
133 switch(gray_state.state)
134 {
135     case 0x0001:gray_status[0]=-11; gray_status_worse/=2;break; //000000000001b
136     case 0x0003:gray_status[0]=-10; gray_status_worse/=2;break; //000000000011b
137     case 0x0002:gray_status[0]=-9; gray_status_worse/=2;break; //000000000010b
138     case 0x0006:gray_status[0]=-8; gray_status_worse/=2;break; //000000000110b
139     case 0x0004:gray_status[0]=-7; gray_status_worse/=2;break; //000000000100b
140     case 0x000C:gray_status[0]=-6; gray_status_worse/=2;break; //0000000001100b
141     case 0x0008:gray_status[0]=-5; gray_status_worse/=2;break; //000000001000b
142     case 0x0018:gray_status[0]=-4; gray_status_worse/=2;break; //000000011000b
143     case 0x0010:gray_status[0]=-3; gray_status_worse/=2;break; //000000010000b
144     case 0x0030:gray_status[0]=-2; gray_status_worse/=2;break; //0000000110000b
145     case 0x0020:gray_status[0]=-1; gray_status_worse/=2;break; //0000000100000b
146     case 0x0060:gray_status[0]=0; gray_status_worse/=2;break; //000001100000b
147     case 0x0040:gray_status[0]=1; gray_status_worse/=2;break; //000001000000b
148     case 0x00C0:gray_status[0]=2; gray_status_worse/=2;break; //000011000000b
149     case 0x0080:gray_status[0]=3; gray_status_worse/=2;break; //000010000000b
150     case 0x0180:gray_status[0]=4; gray_status_worse/=2;break; //000110000000b
151     case 0x0100:gray_status[0]=5; gray_status_worse/=2;break; //000100000000b
152     case 0x0300:gray_status[0]=6; gray_status_worse/=2;break; //001100000000b
153     case 0x0200:gray_status[0]=7; gray_status_worse/=2;break; //001000000000b
154     case 0x0600:gray_status[0]=8; gray_status_worse/=2;break; //011000000000b
155     case 0x0400:gray_status[0]=9; gray_status_worse/=2;break; //010000000000b
156     case 0x0C00:gray_status[0]=10; gray_status_worse/=2;break; //110000000000b
157     case 0x0800:gray_status[0]=11; gray_status_worse/=2;break; //100000000000b
158     case 0x0000:gray_status[0]=gray_status_backup[0][0];gray_status_worse++;break; //00000000b
159     default://其它特殊情况单独判断
160     {
161         gray_status[0]=gray_status_backup[0][0];
162         gray_status_worse++;
163     }
164 }
165
```

以 10mm 宽度的线为例，正常情况下，当 12 路灰度传感器下方存在黑色引导线时，最多只有两路灰度传感器状态会受到黑色引导线影响，即最多存在 2 个灰度传感器输出高电平 1；在没有丢线的情况下至少会有一路灰度传感器输出高电平，基于上述原理可以设计偏差量化过程如下：

12路灰度传感器阵列偏差映射表

bit12	bit11	bit10	bit9	bit8	bit7	bit6	bit5	bit4	bit3	bit2	bit1	二进制	state值	偏差值
0	0	0	0	0	0	0	0	0	0	0	1	000000000001b	0x0001	-11
0	0	0	0	0	0	0	0	0	0	1	1	000000000011b	0x0003	-10
0	0	0	0	0	0	0	0	0	0	1	0	000000000010b	0x0002	-9
0	0	0	0	0	0	0	0	0	1	1	0	000000000110b	0x0006	-8
0	0	0	0	0	0	0	0	0	1	0	0	000000000100b	0x0004	-7
0	0	0	0	0	0	0	0	1	1	0	0	000000001100b	0x000C	-6
0	0	0	0	0	0	0	0	1	0	0	0	000000001000b	0x0008	-5
0	0	0	0	0	0	0	1	1	0	0	0	000000011000b	0x0018	-4
0	0	0	0	0	0	0	1	0	0	0	0	000000010000b	0x0010	-3
0	0	0	0	0	0	1	1	0	0	0	0	000000110000b	0x0030	-2
0	0	0	0	0	0	1	0	0	0	0	0	000000100000b	0x0020	-1
0	0	0	0	0	1	1	0	0	0	0	0	000001100000b	0x0060	0
0	0	0	0	0	1	0	0	0	0	0	0	000001000000b	0x0040	1
0	0	0	0	1	1	0	0	0	0	0	0	000011000000b	0x00C0	2
0	0	0	0	1	0	0	0	0	0	0	0	000010000000b	0x0080	3
0	0	0	1	1	0	0	0	0	0	0	0	000110000000b	0x0180	4
0	0	0	1	0	0	0	0	0	0	0	0	000100000000b	0x0100	5
0	0	1	1	0	0	0	0	0	0	0	0	001100000000b	0x0300	6
0	0	1	0	0	0	0	0	0	0	0	0	001000000000b	0x0200	7
0	1	1	0	0	0	0	0	0	0	0	0	011000000000b	0x0600	8
0	1	0	0	0	0	0	0	0	0	0	0	010000000000b	0x0400	9
1	1	0	0	0	0	0	0	0	0	0	0	110000000000b	0x0C00	10
1	0	0	0	0	0	0	0	0	0	0	0	100000000000b	0x0800	11

```
2 void gpio_input_check_channel_7(void)
3 {
4     gray_state.gray.bit1=GPIOPinRead(GPIO_PORTC_BASE,GPIO_PIN_6)!=0?0x01:0x00;
5     gray_state.gray.bit2=GPIOPinRead(GPIO_PORTC_BASE,GPIO_PIN_7)!=0?0x01:0x00;
6     gray_state.gray.bit3=GPIOPinRead(GPIO_PORTB_BASE,GPIO_PIN_5)!=0?0x01:0x00;
7     gray_state.gray.bit4=GPIOPinRead(GPIO_PORTB_BASE,GPIO_PIN_4)!=0?0x01:0x00;
8
9     gray_state.gray.bit5 =GPIOPinRead(GPIO_PORTD_BASE,GPIO_PIN_0)!=0?0x01:0x00;
10    gray_state.gray.bit6=GPIOPinRead(GPIO_PORTD_BASE,GPIO_PIN_1)!=0?0x01:0x00;
11    gray_state.gray.bit7=GPIOPinRead(GPIO_PORTD_BASE,GPIO_PIN_2)!=0?0x01:0x00;
12
13    for(uint16_t i=19;i>0;i--)
14    {
15        gray_status_backup[0][i]=gray_status_backup[0][i-1];
16    }
17    gray_status_backup[0][0]=gray_status[0];
18    switch(gray_state.state)
19    {
20        case 0x0001:gray_status[0]=-7; gray_status_worse/=2;break; //0000001b
21        case 0x0003:gray_status[0]=-5; gray_status_worse/=2;break; //0000011b
22        case 0x0002:gray_status[0]=-4; gray_status_worse/=2;break; //0000010b
23        case 0x0006:gray_status[0]=-3; gray_status_worse/=2;break; //0000110b
24        case 0x0004:gray_status[0]=-2; gray_status_worse/=2;break; //0000100b
25        case 0x000C:gray_status[0]=-1; gray_status_worse/=2;break; //0001100b
26        case 0x0008:gray_status[0]=0; gray_status_worse/=2;break; //0001000b
27        case 0x0018:gray_status[0]=1; gray_status_worse/=2;break; //0011000b
28        case 0x0010:gray_status[0]=2; gray_status_worse/=2;break; //0010000b
29        case 0x0030:gray_status[0]=3; gray_status_worse/=2;break; //0110000b
30        case 0x0020:gray_status[0]=4; gray_status_worse/=2;break; //0100000b
31        case 0x0060:gray_status[0]=5; gray_status_worse/=2;break; //1100000b
32        case 0x0040:gray_status[0]=7; gray_status_worse/=2;break; //1000000b
33        case 0x0000:gray_status[0]=gray_status_backup[0][0];gray_status_worse++;break; //0000000b
34        default:
35    }
```

7 路灰度传感器阵列检测程序

用户在自己项目中设计偏差量化过程时，需要根据实际引导线的宽度、灰度传感器的间距，结合照射到引导线上的灰度传感器数量信息，对偏差量化过程进行灵活调整。